

# 告别单打独斗！

架构师+AI Agent的打包脚本迭代开发心法

---



分享人：如意玲珑开源社区 Ziggy Luk |

- 01 闲聊: AI正在改变开源社区
- 02 项目背景: 如意玲珑生态打包智能体
- 03 方法论: AI辅助开发任务流程五步法
- 04 实战分享: AI开发立项Sessions
- 05 成效简介

01

闲聊

AI正在改变开源社区

## | 社区涌现的AI开发案例

最近 deepin 社区涌现了一批 AI 开发案例，它们的共同特点：作者并非传统“全栈大佬”，而是借助 AI 工具快速将想法落地。

### DSearch

基于 deepin-anything 的文件搜索工具，  
从底层文件索引到前端搜索 UI 完整链路

### 天气 APP

"这可能是 Linux 下最漂亮的天气 APP"，  
AI 辅助从 UI 设计到实现全程跑通

### Ligocut 光剪

新增对龙芯新旧世界、arm 架构鲲鹏支持，  
跨架构适配效率大幅提升

 核心趋势：AI 正在将“有想法但缺技术”的门槛大幅降低

# 02

## 项目背景

如意玲珑生态打包智能体

# | 如意玲珑与打包智能体

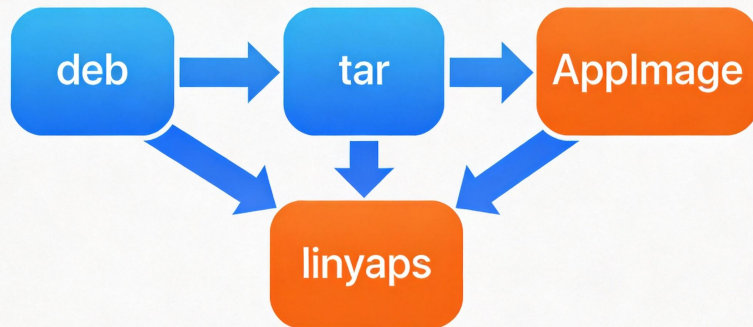
如意玲珑 (**Linyaps**) —— 新一代 Linux 软件分发解决方案

## 核心目标

批量将不同格式的软件包转换为玲珑便捷打包脚本，降低打包门槛

- ✓ deb → linyaps **已实现**
- ✓ binaryTar → linyaps **已实现**

+ 新增需求: **ApplImage 格式支持** —— 当前 agent 仍需追加的功能



# Agent 维护模式: 人类架构师 + AI Agent

libaryaps-app-packaging-script-generator

## 类负责

- 架构设计
- 需求定义
- 质量把关

## 负责

- 代码实现
- 模式分析
- 方案推演



## 内部 Skill 协作

deb-analysis

linglong-project-gen

resource-collector

其他专业 skill

关键优势: 每个 skill 专注自身领域, 人类架构师协调关系

## | 参考项目：linglong-pica

### ✔ 参考价值

ll-pica 已有 ApplImage 转换能力

- ▶ 使用 wrapper 脚本隔离 LD\_LIBRARY\_PATH
- ▶ ApplImage 内容全部放入 lib/ 子目录，保持原始结构
- ▶ 从文件名提取版本号、从 desktop 文件推导包名

**关键洞察：**binaryTar 和 ApplImage 有不少相似之处——都是二进制分发、都需要 wrapper 机制、都需要处理 desktop 文件

### ⚠ 局限性 & 策略

ll-pica 的局限

- ✗ 只支持 deb、appimage、flatpak
- ✗ 对非标准 tar 格式缺乏方案

我们的策略：参考 ll-pica 的 ApplImage 处理经验 + 结合已有 binaryTar-lynyaps 实现，创建新的 **ApplImage-lynyaps** 子 SKILL



# 03

## 方法论

AI辅助开发任务流程五步法

## 五步法详解

1

### 确认目标

目标越具体、越可验证，AI 输出质量越高。例如"参考 tar-linyaps 的 SKILL.md，为 ApplImage 格式创建完整子 SKILL"

2

### 提供参考资料

给 AI 看代码风格、架构设计、已有实现，才能输出符合项目风格的代码

3

### LLM 返回推理规划

AI 分析参考资料，识别模式与架构，输出分阶段实施计划。**关键：人类需审核计划合理性**

4

### 适时调整功能点

降低宽泛目标带来的非必要效率损失。例如 AI 建议扁平化 usr/，但分析后发现 wrapper 机制可复用，调整方案

5

### 开始具体实现

AI 根据确认后的计划逐步实现，人类持续审核和调整

“核心心法：架构师的价值在于**"问对问题"**，AI 的价值在于**"快速验证"**

# 04

## 实战分享

AI开发立项Sessions实战

## | 开发 Session 全景

整个开发过程分为 7 个 session：前 5 个 bug 修复，后 2 个功能开发

| 序号 | 日期   | Session 主题                       | 类型     |
|----|------|----------------------------------|--------|
| 1  | 5/26 | binary name 查找逻辑问题               | Bug 修复 |
| 2  | 5/26 | binary_name 处理漏洞分析               | Bug 修复 |
| 3  | 5/26 | deb 和 tar 模块初始化问题                | Bug 修复 |
| 4  | 5/26 | origin_version 处理问题              | Bug 修复 |
| 5  | 6/02 | deb 资源收集问题                       | Bug 修复 |
| 6  | 6/03 | ll-pica appimage-to-linyaps 实现分析 | 功能分析   |
| 7  | 6/04 | Appimage 业务实现计划制定                | 功能规划   |

→ 重点讲解 Session 6 和 Session 7 两个功能开发 session

## | Session 6: ll-pica 实现分析

目标：分析 linglong-pica 对 AppImage 的转换实现

AI 发现 ll-pica 中存在两套并行的 AppImage 转 Linyaps 实现：

### 🔗 方案一：Go 版本 (ll-appimage-convert)

- 使用 wrapper 脚本隔离 LD\_LIBRARY\_PATH
- AppImage 内容全部放入 lib/\${APP\_PREFIX}/ 子目录
- 保持 squashfs-root 原始结构不变
- 设计更优雅，不需修改内部文件

### 📁 方案二：.ab 脚本版本 (ll-convert-tool)

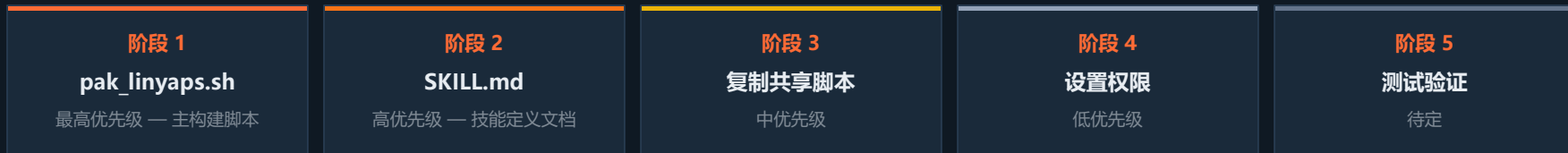
- 扁平化 usr/ 目录
- 修复 AppRun 和 shell 脚本中的路径
- 复杂度更高

### 💡 关键洞察

通过 wrapper 相对路径执行原有 Exec，无需做路径修改

# Session 7: ApplImage 业务实现计划

目标：基于分析结果，制定完整的 **ApplImage SKILL** 实现计划



## 关键设计决策

- srcType 设为 **"applimage"**，区分不同包类型

- wrapper 模式：**cd lib/\${APP\_PREFIX} && ./AppRun \$@**

- 保持 squashfs-root 原样，**不扁平化 usr/**

- Exec 解析优先级：desktop 文件 → AppRun 推导 → **scan\_executables.sh** 兜底

# | 人类架构师的贡献：Bug 修复案例

## 案例一 binary\_name 处理漏洞

### 问题

wrapper 生成前未检测 binary\_name 二进制匹配性，可能指向损坏软链或错误架构 ELF

### 人类架构师贡献

1. 发现问题 — 实际使用中发现 wrapper 指向错误架构二进制
2. 定义问题边界 — 验证 ELF 架构匹配性 + 兼容 shell 脚本
3. 审核方案 — 提醒注意损坏软链和非 ELF 文件边界情况
4. 验证效果 — 用多架构 ELF 和 shell 脚本的 deb 包测试

## 案例二 deb 资源收集问题

### 问题

图标只存在于 pixmaps 目录时，未自动复制到 icons/hicolor 目录

### 人类架构师贡献

1. 发现问题 — 测试发现打包后应用无图标
2. 定义迁移规则 — SVG→scalable, XPM→48x48, PNG 推断尺寸
3. 审核方案 — 补充向后兼容需求，保留 pixmaps 目录
4. 验证效果 — SVG/XPM/PNG 正确迁移且保留原目录

# 协作模式总结

## 👤 人类架构师的价值

- ▶ 发现问题：通过实际使用感知 bug 存在
- ▶ 定义问题边界：明确影响范围和修复约束
- ▶ 提供领域知识：业务规则和向后兼容需求
- ▶ 审核方案：评估修复方案的合理性
- ▶ 验证效果：通过测试用例确认修复符合预期

## 🤖 AI Agent 的价值

- ▶ 快速定位：在大量代码中快速找到问题根因
- ▶ 方案设计：基于上下文设计合理修复方案
- ▶ 代码实现：高效完成修复代码编写
- ▶ 模式识别：识别共性问题提供通用解决方案



二 人类负责"做什么"和"对不对", AI 负责"怎么做"和"快不快"



05

## 成效简介

里程碑达成

# | 里程碑达成

## 项目数据

3

支持格式  
deb · tar · Appliance

700+

pak\_linyaps.sh  
行数 × 3 个版本

10+

核心脚本  
模块化设计

3

Appliance 验证通  
过  
x86\_64 + aarch64

## 典型使用场景

### 批量打包

提供 CSV/JSON 配置，自动批量转换

### 单包处理

交互式引导，逐步完成转换

### 问题修复

自动检测并修复常见问题

深度科技社区



微信公众号



[github.com/OpenAtom-Linyaps/linyaps-app-packaging-script-generator](https://github.com/OpenAtom-Linyaps/linyaps-app-packaging-script-generator)